



FORMATO DE TAREA

I. PORTADA

UNIVERSIDAD TÉCNICA DE AMBATO



Facultad de Ingeniería en Sistemas, Electrónica e Industrial

CARRERA DE SOFTWARE

Título:	Recorridos de Árboles Binarios
Carrera:	Software
Unidad de Organización Curricular:	Básico
Línea de Investigación:	Sistemas de Información
Nivel y Paralelo:	3do "B"
Alumnos participantes:	Nata Analuiza Walter Alexis
Módulo y Docente:	Estructura de Datos. Ing José Caiza Mg.



II. INFORME

2.1 Título

Recorridos de Árboles Binarios

2.2 Objetivo

s

General:

Implementar y analizar los principales recorridos de árboles binarios utilizando C++ y Java, aplicando estructuras de datos dinámicas, recursividad y colas.

Objetivos Específicos:

- Desarrollar la estructura de un árbol binario en C++ y Java, comprendiendo cómo se organizan y enlazan los nodos mediante memoria dinámica para representar jerarquías de datos.
- Aplicar los distintos tipos de recorridos de árboles binarios (preorden, inorden y postorden) utilizando recursividad, así como el recorrido por niveles (BFS) mediante el uso de una cola, con el fin de analizar su comportamiento y diferencias.
- Relacionar el uso de los recorridos de árboles binarios con un caso práctico, permitiendo interpretar su utilidad en la organización y procesamiento de información dentro de un sistema real.

2.3 Introducción

Las estructuras de datos son fundamentales en el desarrollo de software, ya que nos permiten organizar y gestionar la información de manera eficiente. Dentro de estas, los árboles binarios destacan por su capacidad para representar datos jerárquicos, facilitando operaciones como búsqueda, inserción y recorrido.

A continuación, vamos a abordar el estudio de los principales recorridos de árboles binarios, específicamente los recorridos en profundidad (DFS), que incluyen preorden, inorden y postorden, así como el recorrido en amplitud (BFS). Cada uno de estos métodos permite visitar los nodos del árbol siguiendo diferentes criterios, lo que resulta útil según el contexto de aplicación.

Para el desarrollo de esta práctica se implementaron estos recorridos utilizando los lenguajes de programación C++ y Java, haciendo uso de conceptos como la recursividad y estructuras auxiliares como la cola. Además, se plantea un caso real que permite evidenciar la aplicación práctica de estos recorridos en un sistema, reforzando así la comprensión de su utilidad en el ámbito profesional.

El proceso de desarrollo fue documentado con el sistema de control de versiones Git, t GitHub, en donde se encuentra el repositorio clonado y los archivos .cpp y .java, cada proceso se documentó mediante commits en el cual describimos la actividad que se realizó en el commit.

2.4 Marco Teórico

2.4.1. Árboles Binarios

Un árbol binario es una estructura de datos compuesta por un nodo raíz y dos subárboles (izquierdo y derecho), útil para organizar información de tamaño variable y acceder a ella ordenadamente. Cada nodo contiene datos y referencias a sus subárboles, permitiendo tres formas principales de recorrido: preorden, inorden y postorden.[1]



2.4.2. **Nodos y estructuras**

Un nodo es la unidad básica de un árbol binario y contiene un valor junto con referencias a sus hijos. El nodo principal se denomina raíz, mientras que los nodos sin hijos se conocen como hojas. La conexión entre nodos permite formar la estructura jerárquica característica del árbol.[2]

2.4.3. **Recorridos de arboles binarios**

El recorrido de un árbol es una operación fundamental que consiste en visitar cada uno de sus nodos para ejecutar una acción específica, lo que permite procesar, buscar o visualizar los datos almacenados en la estructura.[3]

2.4.4. **Recorrido en profundidad (DFS)**

El recorrido DFS (Depth First Search) consiste en explorar cada rama del árbol hasta llegar al nodo más profundo antes de retroceder. Este tipo de recorrido se implementa generalmente mediante recursividad y se divide en:[4]

- **Preorden:** primero la raíz, luego el subárbol izquierdo y finalmente el derecho.
- **Inorden:** primero el subárbol izquierdo, luego la raíz y finalmente el derecho.
- **Postorden:** primero el subárbol izquierdo, luego el derecho y por último la raíz.

2.4.5. **Recorrido en amplitud (BFS)**

El recorrido BFS (Breadth First Search) visita los nodos nivel por nivel, comenzando desde la raíz. Para su implementación se utiliza una estructura auxiliar llamada cola, que permite mantener el orden de los nodos pendientes por visitar.[5]

2.4.6. **Recursividad**

La recursividad es una técnica de programación en la cual una función se llama a sí misma para resolver un problema. En los árboles binarios, se utiliza principalmente en los recorridos DFS, permitiendo simplificar la lógica al dividir el problema en subárboles.[6]

2.4.7. **Estructuras auxiliares: cola**

La cola es una estructura de datos que sigue el principio FIFO (First In, First Out). En el recorrido BFS, se utiliza para almacenar temporalmente los nodos que deben ser visitados, asegurando un recorrido por niveles.[7]

2.4.8. **Implementacion en C++ y Java**

Los árboles binarios pueden implementarse en distintos lenguajes de programación. En C++, se utiliza memoria dinámica, lo que requiere gestionar manualmente la liberación de memoria. En Java, en cambio, la gestión de memoria es automática gracias al recolector de basura, lo que simplifica el manejo de los objetos.

2.5 Herramientas utilizadas

2.5.1 **Lenguaje C++:**

Se utilizó para implementar los recorridos de árboles binarios trabajando directamente con memoria dinámica.

2.5.2 **Lenguaje Java:**

Se empleó para desarrollar la misma lógica de los recorridos, pero con un enfoque más orientado a objetos. A diferencia de C++, Java facilita la gestión de memoria mediante su recolector automático, lo que simplifica el



trabajo con estructuras como árboles.

2.5.3 Visual Studio Code:

Fue el entorno de desarrollo utilizado para escribir y ejecutar el código. Su interfaz sencilla y sus extensiones permiten trabajar de manera cómoda tanto con C++ como con Java en un mismo proyecto.

2.5.4 Git:

Se utilizó para llevar un control ordenado de los cambios realizados durante el desarrollo, permitiendo guardar versiones del proyecto y evitar la pérdida de información.

2.5.5 GitHub:

Se empleó como plataforma para almacenar el repositorio en línea, facilitando el acceso al proyecto y la entrega del trabajo de forma organizada.

2.5.6 GitBash:

Se utilizó como herramienta de apoyo para ejecutar comandos de Git desde la terminal, permitiendo gestionar el repositorio de manera más directa y eficiente.

2.5.7 Compilador g++:

Utilizado para compilar y ejecutar el código en C++, permitiendo transformar el código fuente en un programa ejecutable.

2.5.8 Java JDK:

Conjunto de herramientas necesarias para compilar y ejecutar programas en Java, incluyendo el compilador javac y la máquina virtual Java (JVM).

2.6 Resultados obtenidos

2.6.1 Creación del repositorio

Durante el desarrollo de la práctica, se logró implementar correctamente los recorridos de árboles binarios tanto en C++ como en Java, cumpliendo con los objetivos planteados. En primer lugar, se creó el repositorio del proyecto en GitHub con el nombre de "Recorrido_Arboles_Binarios". Esto permitió organizar el código fuente, la documentación y evidencias del trabajo de manera estructurada. Donde también cuenta con el archivo .gitignore que evita que el repositorio suba carpetas o archivos que llenen el repositorio, al igual que el README.md, donde se explica sobre el repositorio y como esta desarrollado.

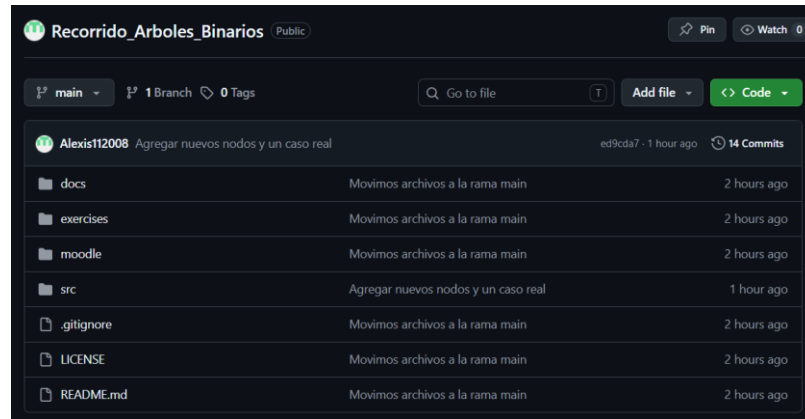


Ilustración 1. Repositorio Recorrido_Arboles_Binarios

2.6.2 Historial de Commits

Historial de commits del repositorio que evidencia el desarrollo progresivo del proyecto y la implementación de los recorridos de árboles binarios. En total hemos desarrollado 14 commits en donde, cada uno tiene una descripción de lo que se realizó en cada cambio.

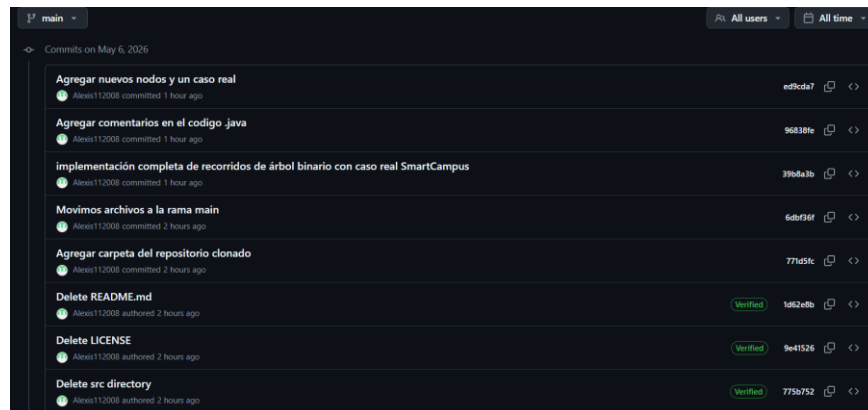


Ilustración 2. Historial de commits

2.6.3 Clonar el Repositorio

Una vez creado el repositorio, vamos a clonar el mismo para poder empezar a trabajar en nuestra tarea, hacemos utilidad del siguiente comando para clonarlo.

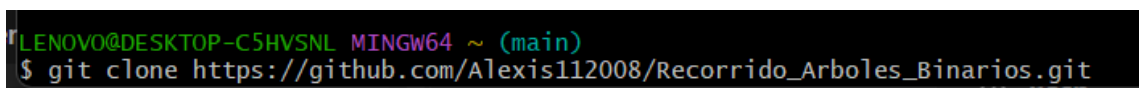


Ilustración 3. Comando para clonar repositorio

Una vez clonado vamos a transferir todas las carpetas del archivo comprimido que se



encuentra en las indicaciones del deber, en esta ocasión vamos hacerlo de forma manual.

Nombre	Tipo	Tamaño	Fecha de modificación
assets	Carpeta de archivos		5/5/2026 10:42
docs	Carpeta de archivos		5/5/2026 10:42
exercises	Carpeta de archivos		5/5/2026 10:42
moodle	Carpeta de archivos		5/5/2026 10:42
src	Carpeta de archivos		5/5/2026 10:42
.gitignore	Archivo de origen Git Ign...	1 KB	5/5/2026 10:42
LICENSE	Archivo	1 KB	5/5/2026 10:42
README	Archivo de origen Markdo...	3 KB	5/5/2026 11:03

Ilustración 4. Estructura de carpetas

Una vez realizado estos pasos entonces comprobamos si los archivos se cargaron correctamente.

```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~ (main)
$ cd Recorrido_Arboles_Binarios

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/Recorrido_Arboles_Binarios (main)
$ ls
LICENSE  README.md  assets/  docs/  exercises/  moodle/  src/

LENOVO@DESKTOP-C5HVSNL MINGW64 ~/Recorrido_Arboles_Binarios (main)
$
```

Ilustración 5. Repositorio remoto

Una vez clonado el repositorio entonces vamos a empezar a trabajar en nuestra tarea siguiendo cada una de las indicaciones, que nos piden, para trabajar en el mismo en GitBash aplicamos el siguiente comando **code** . con este se abrirá nuestro VS code y empezamos a trabajar.

2.6.4 Estructura de la tarea

La estructura del proyecto se organiza de manera modular para facilitar su comprensión y mantenimiento. En la carpeta `src/` se encuentran las implementaciones principales, separadas por lenguaje (`cpp/` para C++ y `java/` para Java), donde se desarrolla la lógica de los recorridos de árboles binarios. Las carpetas `docs/`, `exercises/` y `moodle/` contienen material de apoyo como guías, ejercicios y preguntas para reforzar el aprendizaje. Se incluyen archivos de configuración como `.gitignore`, la licencia del proyecto y el archivo `README.md`, que documenta toda la información general.

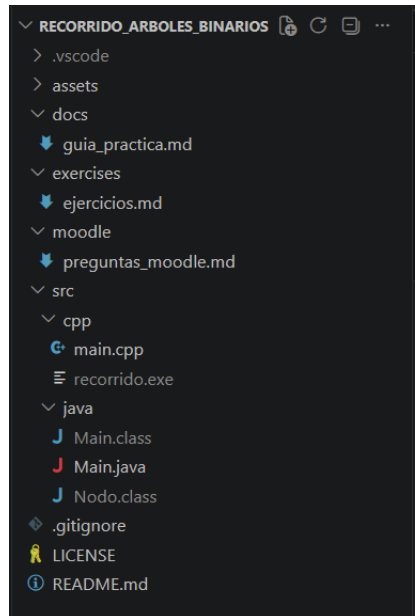


Ilustración 6. Estructura de carpetas

2.6.5 Historial de commits en GitBash

Se utilizó el comando git log --oneline para visualizar el historial de commits del repositorio, permitiendo observar de manera resumida los cambios realizados durante el desarrollo del proyecto. Esto facilitó el seguimiento de la evolución del código y la organización de las distintas etapas de implementación.

```
LENOVO@DESKTOP-C5HVSNL MINGW64 ~/Recorrido_Arboles_Binarios (main)
$ git log
commit ed9cda770c966c7cc98da4f7d235829e3e3f7801 (HEAD -> main, origin/main, origin/HEAD)
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Wed May 6 09:49:28 2026 -0500

    Agregar nuevos nodos y un caso real

commit 96838fed31d173674247e0943c3e37e8e3389bf2
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Wed May 6 09:45:39 2026 -0500

    Agregar comentarios en el codigo .java

commit 39b8a3bffc1e574bb9a183c1ac6b7f3a9be556d5
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Wed May 6 09:39:16 2026 -0500

    implementación completa de recorridos de árbol binario con caso real SmartCampus

commit 6dbf36fb4c8c81bdb3fc383cff559fa06c65c202
Author: Alexis Nata <alexis2000nata@gmail.com>
Date: Wed May 6 08:24:21 2026 -0500

    Movimos archivos a la rama main

commit 771d5fc945d6592148dc6066d3b69e0b93a8d240
Author: Alexis Nata <alexis2000nata@gmail.com>
commit 17982e1935f37378c4d3d119e85fd270ce5943a6
Author: Alexis112008 <alexis2000nata@gmail.com>
Date: Wed May 6 08:17:01 2026 -0500

    Delete docs directory
```

Ilustración 7. Historial de commits

2.6.6 Implementación de los recorridos en Código C++

Se define una estructura llamada Nodo, la cual representa cada elemento del árbol binario. Cada nodo contiene un valor entero (dato) y dos punteros (izquierda y derecha) que apuntan a sus hijos dentro del árbol.



El constructor inicializa el nodo con un valor específico y establece los punteros en nullptr, indicando que inicialmente no tiene hijos.

```
struct Nodo {  
    int dato;  
    Nodo* izquierda;  
    Nodo* derecha;  
  
    // Constructor: inicializa  
    Nodo(int valor) {  
        dato = valor;  
        izquierda = nullptr;  
        derecha = nullptr;  
    }  
};
```

Ilustración 8. Estructura del Nodo

Se implemento los siguientes recorridos:

Preorden: En este caso se implementa de forma recursiva y sirve para representar la jerarquía del árbol o realizar copias de estructuras.

Inorden: Este recorrido visita primero el subárbol izquierdo, luego la raíz y después el subárbol derecho. Se usa principalmente para obtener los datos del árbol en un orden lógico.

Postorden: En este caso se recorre primero el subárbol izquierdo, luego el derecho y al final la raíz.

```
// PREORDEN → Raíz - Izquierda - Derecha  
void preorden(Nodo* raiz) {  
    if (raiz == nullptr) return;  
    cout << raiz->dato << " ";  
    preorden(raiz->izquierda);  
    preorden(raiz->derecha);  
}  
  
// INORDEN → Izquierda - Raíz - Derecha  
void inorden(Nodo* raiz) {  
    if (raiz == nullptr) return;  
    inorden(raiz->izquierda);  
    cout << raiz->dato << " ";  
    inorden(raiz->derecha);  
}  
  
// POSTORDEN → Izquierda - Derecha - Raíz  
void postorden(Nodo* raiz) {  
    if (raiz == nullptr) return;  
    postorden(raiz->izquierda);  
    postorden(raiz->derecha);  
    cout << raiz->dato << " ";  
}
```

Ilustración 9. Recorridos

Recorrido BFS

Este recorrido visita el árbol por niveles, es decir, primero la raíz, luego todos los nodos del siguiente nivel y así sucesivamente. Para su implementación se utiliza una estructura de cola (queue), que permite procesar los nodos en orden FIFO (primero en entrar, primero en salir). En este caso, el BFS sirve para recorrer el árbol de manera amplia, útil para explorar estructuras completas o encontrar caminos de forma eficiente en cada nivel del árbol.



```
// BFS (Breadth-First Search) → Nivel por nivel, utilizado para encontrar
void bfs(Nodo* raiz) {
    if (raiz == nullptr) return;    // Si el árbol está vacío, no hacemos nada

    queue<Nodo*> cola;               // Creamos una cola para almacenar los nodos
    cola.push(raiz);               // Comenzamos con la raíz

    while (!cola.empty()) {        // Mientras haya nodos en la cola, seguimos
        Nodo* actual = cola.front(); // Obtenemos el nodo al frente de la cola
        cola.pop();

        cout << actual->dato << " "; // Imprimimos el dato del nodo actual

        // Agregamos los hijos del nodo actual a la cola para procesarlos
        if (actual->izquierda != nullptr) cola.push(actual->izquierda);
        if (actual->derecha != nullptr) cola.push(actual->derecha);
    }
}
```

Ilustración 10.Recorrido BFS

Caso real: SmartCampus

En esta sección se modela un sistema real de gestión de trámites estudiantiles de la Universidad Técnica de Ambato (SmartCampus UTA), utilizando un árbol binario. Cada nodo representa un tipo de trámite académico o financiero, como matrículas, retiros, becas y pagos. La estructura permite organizar jerárquicamente los procesos administrativos de la institución. Finalmente, se aplican los cuatro recorridos del árbol (Preorden, Inorden, Postorden y BFS) para analizar la estructura desde diferentes perspectivas.

```
// CASO REAL - SmartCampus UTA
void casoRealSmartCampus() {
    cout << "-----" << endl;
    cout << " CASO REAL - SmartCampus UTA" << endl;
    cout << " Arbol de Tramites Estudiantiles" << endl;
    cout << "-----" << endl;

    // Nodos base (7 originales)
    Nodo* tramites = new Nodo(1); // Raíz: Trámites
    Nodo* academicos = new Nodo(2); // Académicos
    Nodo* financieros = new Nodo(3); // Financieros
    Nodo* matricula = new Nodo(4); // Matrícula
    Nodo* retiro = new Nodo(5); // Retiro de materia
    Nodo* beca = new Nodo(6); // Becas
    Nodo* pago = new Nodo(7); // Pagos

    // 5 nodos adicionales (actividad)
    Nodo* cambioCarrera = new Nodo(8); // Cambio de carrera
    Nodo* convalidacion = new Nodo(9); // Convalidación
    Nodo* apelacion = new Nodo(10); // Apelación de nota
    Nodo* revision = new Nodo(11); // Revisión de examen
    Nodo* certificado = new Nodo(12); // Certificado de estudios

    // Construcción del árbol
    tramites->izquierda = academicos;
    tramites->derecha = financieros;
    academicos->izquierda = matricula;
    academicos->derecha = retiro;
    financieros->izquierda = beca;
    financieros->derecha = pago;
}
```



```
// Nodos adicionales
matricula->izquierda = cambioCarrera;
matricula->derecha = convalidacion;
convalidacion->derecha = apelacion;
apelacion->izquierda = revision;
beca->izquierda = certificado;

// Leyenda de referencia
cout << "\nLEYENDA DE NODOS:" << endl;
cout << " 1 = Tramites  2 = Academicos  3 = Financieros" << endl;
cout << " 4 = Matricula  5 = Retiro      6 = Beca" << endl;
cout << " 7 = Pago       8 = CambioCarr  9 = Convalidacion" << endl;
cout << "10 = Apelacion 11 = Revision 12 = Certificado" << endl;

cout << "\nPreorden (jerarquía): ";
preorden(tramites);

cout << "\nInorden (orden proc): ";
inorden(tramites);

cout << "\nPostorden (liberar mem): ";
postorden(tramites);

cout << "\nBFS (nivel x nivel): ";
bfs(tramites);
cout << endl;
```

La función main

Es el punto de inicio del programa. Aquí se construye un árbol binario de forma manual, asignando nodos y sus respectivas conexiones izquierda y derecha.

Primero se crea un árbol base con valores numéricos, y luego se agregan nodos adicionales para hacerlo más complejo y representar mejor la estructura jerárquica.

```
cout << "-----" << endl;
cout << " Recorrido de arboles binarios" << endl;
cout << "-----" << endl;

Nodo* raiz = new Nodo(10);
raiz->izquierda = new Nodo(5);
raiz->derecha = new Nodo(15);
raiz->izquierda->izquierda = new Nodo(2);
raiz->izquierda->derecha = new Nodo(7);
raiz->derecha->izquierda = new Nodo(12);
raiz->derecha->derecha = new Nodo(20);

// 5 nodos adicionales
raiz->izquierda->izquierda->izquierda = new Nodo(1); // Nodo adicional 1
raiz->izquierda->izquierda->derecha = new Nodo(3); // Nodo adicional 2
raiz->derecha->izquierda->derecha = new Nodo(13); // Nodo adicional 3
raiz->derecha->izquierda->derecha->derecha = new Nodo(14); // Nodo adicional 4
raiz->derecha->izquierda->derecha->derecha->izquierda = new Nodo(11); // Nodo adicional 5
```

Ilustración 11.Main

- Después, se ejecutan los cuatro recorridos del árbol:
- Preorden, Inorden, Postorden, BFS

Esto permite analizar el comportamiento del árbol desde diferentes formas de recorrido.



```
cout << "RECORRIDOS DE ARBOLES BINARIOS - UTA" << endl;

cout << "Preorden: ";
preorden(raiz);

cout << "\nInorden: ";
inorden(raiz);

cout << "\nPostorden: ";
postorden(raiz);

cout << "\nBFS: ";
bfs(raiz);

cout << endl;
```

Ilustración 12.Recorridos

Finalmente, se ejecuta el caso real “SmartCampus UTA”, donde se aplica la misma lógica a un sistema de trámites estudiantiles.

```
// caso real SmartCampus UTA
casoRealSmartCampus();

return 0;
}
```

Ilustración 13.Caso Real

2.6.7 Implementación del recorrido en Java

En Java se define una clase Nodo. A través del constructor, se inicializa el dato del nodo y se establecen sus referencias en null, indicando que inicialmente no tiene conexiones. Esta implementación permite trabajar con árboles binarios.

```
// Nodo de un árbol binario
class Nodo {
    int dato;
    Nodo izquierda;
    Nodo derecha;

    // Constructor
    public Nodo(int dato) {
        this.dato = dato;
        this.izquierda = null;
        this.derecha = null;
    }
}
```

Ilustración 14.Estructura de Nodos

Recorridos DFS

Preorden: Se utiliza cuando se necesita visitar los nodos desde el nivel superior hacia abajo,



manteniendo la jerarquía del árbol.

Inorden: Es muy utilizado en árboles binarios de búsqueda porque permite obtener los valores en orden ascendente.

Postorden: Este método es útil cuando se necesita eliminar o evaluar nodos, ya que se procesan primero los hijos antes que el padre.

```
// Recorrido en preorden: Raíz -> Izquierda -> Derecha
public static void preorden(Nodo raiz) {
    if (raiz == null) return;
    System.out.print(raiz.dato + " ");
    preorden(raiz.izquierda);
    preorden(raiz.derecha);
}

// Recorrido en inorden: Izquierda -> Raíz -> Derecha
public static void inorden(Nodo raiz) {
    if (raiz == null) return;
    inorden(raiz.izquierda);
    System.out.print(raiz.dato + " ");
    inorden(raiz.derecha);
}

public static void postorden(Nodo raiz) {
    if (raiz == null) return;
    postorden(raiz.izquierda);
    postorden(raiz.derecha);
    System.out.print(raiz.dato + " ");
}
```

Ilustración 15. Recorridos

Recorrido BFS

Este recorrido visita el árbol nivel por nivel, empezando desde la raíz y avanzando hacia los niveles inferiores. Para su implementación se utiliza una cola (Queue), lo que permite procesar los nodos en orden de llegada (FIFO). Este método es útil para recorrer estructuras amplias y analizar el árbol por niveles.

```
// Recorrido en anchura (BFS): Nivel por nivel
public static void bfs(Nodo raiz) {
    if (raiz == null) return;

    Queue<Nodo> cola = new LinkedList<>(); // Cola para almacenar los nodos
    cola.add(raiz); // Agregar el nodo raíz a la cola

    while (!cola.isEmpty()) { // Mientras haya nodos en la cola
        Nodo actual = cola.poll(); // Obtener el siguiente nodo de la cola
        System.out.print(actual.dato + " "); // Procesar el nodo actual

        // Agregar los hijos del nodo actual a la cola
        if (actual.izquierda != null) cola.add(actual.izquierda);
        if (actual.derecha != null) cola.add(actual.derecha);
    }
}
```

Ilustración 16. recorrido BFS

Caso Real SmartCampus Java

En esta parte del programa se modela un sistema real de trámites estudiantiles de la Universidad Técnica de Ambato (SmartCampus UTA) utilizando un árbol binario. Esta es la misma implementación que la de en C++.



```
// CASO REAL - SmartCampus UTA
public static void casoRealSmartCampus() {
    System.out.println(x: "\n-----");
    System.out.println(x: " CASO REAL - SmartCampus UTA");
    System.out.println(x: " Arbol de Tramites Estudiantiles");
    System.out.println(x: "-----");

    // Nodos base
    Nodo tramites = new Nodo(dato: 1);
    Nodo academicos = new Nodo(dato: 2);
    Nodo financieros = new Nodo(dato: 3);
    Nodo matricula = new Nodo(dato: 4);
    Nodo retiro = new Nodo(dato: 5);
    Nodo beca = new Nodo(dato: 6);
    Nodo pago = new Nodo(dato: 7);

    // 5 nodos adicionales
    Nodo cambioCarrera = new Nodo(dato: 8);
    Nodo convalidacion = new Nodo(dato: 9);
    Nodo apelacion = new Nodo(dato: 10);
    Nodo revision = new Nodo(dato: 11);
    Nodo certificado = new Nodo(dato: 12);

    // Construcción del árbol
    tramites.izquierda = academicos;
    tramites.derecha = financieros;
    academicos.izquierda = matricula;
    academicos.derecha = retiro;
    financieros.izquierda = beca;
    financieros.derecha = pago;

    // Nodos adicionales
    matricula.izquierda = cambioCarrera;
    matricula.derecha = convalidacion;
    convalidacion.derecha = apelacion;
    apelacion.izquierda = revision;
    beca.izquierda = certificado;

    System.out.println(x: "\nLEYENDA DE NODOS:");
    System.out.println(x: " 1 = Tramites  2 = Academicos  3 = Financieros");
    System.out.println(x: " 4 = Matricula 5 = Retiro      6 = Beca");
    System.out.println(x: " 7 = Pago      8 = CambioCarr  9 = Convalidacion");
    System.out.println(x: "10 = Apelacion 11 = Revision  12 = Certificado");

    System.out.print(s: "\nPreorden (jerarquia): ");
    preorden(tramites);

    System.out.print(s: "\nInorden (orden proc): ");
    inorden(tramites);

    System.out.print(s: "\nPostorden (liberar mem): ");
    postorden(tramites);

    System.out.print(s: "\nBFS (nivel x nivel): ");
    bfs(tramites);
    System.out.println();
}
```

Es el punto de inicio del programa en Java. Se imprime un encabezado para identificar el tema del proyecto. Se construye un árbol binario de ejemplo de forma manual, asignando nodos y sus conexiones (izquierda y derecha). Se utiliza el valor de cada nodo para representar la estructura del árbol.



```
Run | Debug
public static void main(String[] args) {

    System.out.println(x: "\n-----");
    System.out.println(x: "  Recorridos de Arboles Binarios");
    System.out.println(x: "-----");

    // Crear un árbol binario de ejemplo
    Nodo raiz = new Nodo(dato: 10); // Nodo raíz con valor 10
    raiz.izquierda = new Nodo(dato: 5); // Hijo izquierdo del nodo raíz con
    raiz.derecha = new Nodo(dato: 15); // Hijo derecho del nodo raíz con va
    raiz.izquierda.izquierda = new Nodo(dato: 2); // Hijo izquierdo del nod
    raiz.izquierda.derecha = new Nodo(dato: 7); // Hijo derecho del nodo 5
    raiz.derecha.izquierda = new Nodo(dato: 12); // Hijo izquierdo del nodo
    raiz.derecha.derecha = new Nodo(dato: 20); // Hijo derecho del nodo 15

    // 5 nodos adicionales
    raiz.izquierda.izquierda.izquierda = new Nodo(dato: 1);
    raiz.izquierda.izquierda.derecha = new Nodo(dato: 3);
    raiz.derecha.izquierda.derecha = new Nodo(dato: 13);
    raiz.derecha.izquierda.derecha.derecha = new Nodo(dato: 14);
    raiz.derecha.izquierda.derecha.derecha.izquierda = new Nodo(dato: 11);
}
```

Ilustración 17.Main

Se ejecutan los recorridos del árbol:

- Preorden, Inorden, Postorden, BFS

```
System.out.println(x: "RECORRIDOS DE ARBOLES BINARIOS - UTA");

System.out.print(s: "Preorden: ");
preorden(raiz);

System.out.print(s: "\nInorden: ");
inorden(raiz);

System.out.print(s: "\nPostorden: ");
postorden(raiz);

System.out.print(s: "\nBFS: ");
bfs(raiz);

System.out.println();
```

Ilustración 18.Recorridos impresión

Cada recorrido muestra el orden en que se visitan los nodos según su algoritmo. Finalmente se ejecuta el caso real “SmartCampus UTA”, aplicando la misma lógica a un sistema de trámites estudiantiles.

```
//caso real SmartCampus UTA
casoRealSmartCampus();
}
```

Ilustración 19.Caso Real

2.6.8 Tabla comparativa



Tabla 1. Comparativa

Criterio	C++	Java
Manejo de memoria	Manual (uso de new y punteros)	Automático (recolección de basura)
Estructura del nodo	struct Nodo con punteros	class Nodo con referencias
Complejidad del código	Más bajo nivel, más detallado	Más estructurado y limpio
Recursividad en recorridos	Igual en todos los recorridos	Igual en todos los recorridos
Implementación BFS	Uso de queue de STL	Uso de Queue con LinkedList
Facilidad de aprendizaje	Más complejo para principiantes	Más fácil y seguro
Control del sistema	Mayor control del hardware	Menor control, más abstracto
Seguridad de memoria	Riesgo de errores (punteros)	Más seguro (sin punteros directos)

Ambas implementaciones son funcionalmente equivalentes en términos de lógica de recorridos. Sin embargo, Java ofrece mayor seguridad y facilidad de manejo de memoria, mientras que C++ proporciona mayor control y eficiencia a nivel del sistema.

2.6.9 Capturas de Pantalla de C++

En esta captura se muestra la ejecución del programa en C++, donde se construye un árbol binario y se aplican los cuatro tipos de recorridos: Preorden, Inorden, Postorden y BFS. Se puede observar cómo cada recorrido visita los nodos en diferente orden según su lógica:

```
-----
Recorrido de arboles binarios
-----
RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 13 14 11 20
Inorden:  1 2 3 5 7 10 12 13 11 14 15 20
Postorden: 1 3 2 7 5 11 14 13 12 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 13 14 11
-----
```

Se muestra la ejecución del caso real SmartCampus UTA en C++, donde se aplica un árbol binario para representar trámites estudiantiles y se ejecutan los recorridos Preorden, Inorden, Postorden y BFS correctamente.



```
-----
CASO REAL - SmartCampus UTA
Arbol de Tramites Estudiantiles
-----

LEYENDA DE NODOS:
 1 = Tramites  2 = Academicos  3 = Financieros
 4 = Matricula 5 = Retiro      6 = Beca
 7 = Pago      8 = CambioCarr  9 = Convalidacion
10 = Apelacion 11 = Revision   12 = Certificado

Preorden (jerarquia): 1 2 4 8 9 10 11 5 3 6 12 7
Inorden (orden proc): 8 4 9 11 10 2 5 1 12 6 3 7
Postorden (liberar mem): 8 11 10 9 4 5 2 12 6 7 3 1
BFS (nivel x nivel): 1 2 3 4 5 6 7 8 9 12 10 11
```

2.6.10 Implementación del código en Java

En esta captura se muestra la ejecución del programa en Java, donde se construye un árbol binario de ejemplo y se aplican los recorridos Preorden, Inorden, Postorden y BFS. Se verifica el correcto funcionamiento de los algoritmos implementados utilizando estructuras de tipo clase y referencias.

```
-----
Recorridos de Arboles Binarios
-----

RECORRIDOS DE ARBOLES BINARIOS - UTA
Preorden: 10 5 2 1 3 7 15 12 13 14 11 20
Inorden: 1 2 3 5 7 10 12 13 11 14 15 20
Postorden: 1 3 2 7 5 11 14 13 12 20 15 10
BFS: 10 5 15 2 7 12 20 1 3 13 14 11
```

En esta captura se presenta el caso real “SmartCampus UTA” implementado en Java, donde se modela un sistema de trámites estudiantiles mediante un árbol binario. Se observa la estructura jerárquica de nodos y la aplicación de los recorridos Preorden, Inorden, Postorden y BFS para analizar la información por diferentes órdenes.

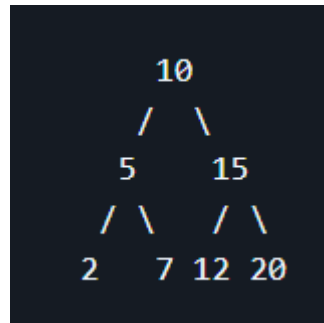
```
-----
CASO REAL - SmartCampus UTA
Arbol de Tramites Estudiantiles
-----

LEYENDA DE NODOS:
 1 = Tramites  2 = Academicos  3 = Financieros
 4 = Matricula 5 = Retiro      6 = Beca
 7 = Pago      8 = CambioCarr  9 = Convalidacion
10 = Apelacion 11 = Revision   12 = Certificado

Preorden (jerarquia): 1 2 4 8 9 10 11 5 3 6 12 7
Inorden (orden proc): 8 4 9 11 10 2 5 1 12 6 3 7
Postorden (liberar mem): 8 11 10 9 4 5 2 12 6 7 3 1
BFS (nivel x nivel): 1 2 3 4 5 6 7 8 9 12 10 11
```

Ejecución de los ejercicios

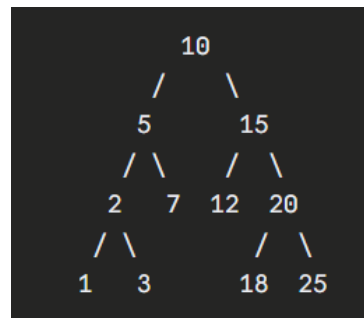
Ejercicio 1:



Escriba manualmente:

- Preorden: 10, 5, 2, 7, 15, 12, 20
- Inorden: 2, 5, 7, 10, 12, 15, 20
- Postorden: 2, 7, 5, 12, 20, 15, 10
- BFS: 10, 5, 15, 2, 7, 12, 20

Ejercicio 2



Modifique el árbol anterior agregando los nodos 1, 3, 18 y 25. Ejecute nuevamente los recorridos.

Al agregar los nodos, el árbol se reorganiza manteniendo las reglas del BST.

Los recorridos cambian según la nueva estructura pero el orden sigue siendo el mismo concepto.

Ejercicio 3:

Implemente una función que cuente la cantidad total de nodos del árbol.

Se cuenta recorriendo todo el árbol y sumando cada nodo visitado.

Es una función recursiva simple que devuelve el total.

```
int contarNodos(Nodo* n) {  
    if (!n) return 0;  
    return 1 + contarNodos(n->izq) + contarNodos(n->der);  
}
```

```
static int contarNodos(Nodo n) {  
    if (n == null) return 0;  
    return 1 + contarNodos(n.izq) + contarNodos(n.der);  
}
```



Ejercicio 4:

Implemente una función que cuente las hojas del árbol.

Se cuentan los nodos que no tienen hijos (hojas).

Se verifica izquierda y derecha en cada nodo.

```
int contarHojas(Nodo* n) {  
    if (!n) return 0;  
    if (!n->izq && !n->der) return 1;           // nodo hoja  
    return contarHojas(n->izq) + contarHojas(n->der);  
}
```

```
static int contarHojas(Nodo n) {  
    if (n == null) return 0;  
    if (n.izq == null && n.der == null) return 1;  
    return contarHojas(n.izq) + contarHojas(n.der);  
}
```

Ejercicio 5:

Explique qué recorrido usaría para:

1. **Mostrar menú principal:** Preorden, porque muestra primero la raíz (Sistema Web).
2. **Procesar módulos internos:** Postorden, porque primero ejecuta los submódulos.
3. **Mostrar nivel por nivel:** BFS, porque recorre el árbol por niveles usando cola.

```
=== EJERCICIO 5: Modulos del sistema web ===  
-- Preorden (menu principal) --  
Sistema Web  
Usuarios  
Registrar  
Buscar  
Inventario  
Productos  
Reportes  
-- Postorden (modulos internos primero) --  
Registrar  
Buscar  
Usuarios  
Productos  
Reportes  
Inventario  
Sistema Web  
-- BFS (nivel por nivel) --  
Sistema Web  
Usuarios  
Inventario  
Registrar  
Buscar  
Productos  
Reportes
```



```
=== EJERCICIO 3: Contar nodos ===  
Total de nodos: 11  
  
=== EJERCICIO 4: Contar hojas ===  
Total de hojas: 6
```

2.7 Conclusiones

- Se logró implementar correctamente los recorridos de árboles binarios (Preorden, Inorden, Postorden y BFS) tanto en C++ como en Java, aplicando estructuras dinámicas y recursividad.
- Se comprobó que cada recorrido tiene un comportamiento diferente al visitar los nodos, lo que permite analizar la estructura del árbol desde distintas perspectivas según la necesidad del problema.
- La aplicación del caso real “SmartCampus UTA” permitió relacionar la teoría con una situación práctica, demostrando el uso de árboles binarios en la organización de información jerárquica.

2.8 Recomendaciones

- Practicar la construcción de árboles binarios más complejos para reforzar la comprensión de los recorridos y su comportamiento.
- Comparar más estructuras de datos como listas, pilas y colas para entender cuándo es más eficiente utilizar árboles binarios.
- Implementar mejoras en el código como liberación de memoria en C++ o interfaces gráficas en Java para hacer la aplicación más completa y profesional.

2.9 Bibliografía

- [1] “LUDA UAM-Azc.” Accessed: May 05, 2026. [Online]. Available: https://aniei.org.mx/paginas/uam/CursoPoo/curso_poo_12.html
- [2] “Estructura de datos - Árboles - Oscar Blancarte - Software Architecture.” Accessed: May 05, 2026. [Online]. Available: <https://www.oscarblancarteblog.com/2014/08/22/estructura-de-datos-arboles/>
- [3] I. S. C. Guadalupe, “Area Académica: Sistemas Computacionales Tema: Recorrido de Árboles Binarios”.
- [4] “DFS - Recorrido en profundidad | Recorridos sobre grafos.” Accessed: May 05, 2026. [Online]. Available: http://163.10.22.82/OAS/recorrido_grafos/dfs__recorrido_en_profundidad.html
- [5] “BFS - Recorrido en amplitud | Recorridos sobre grafos.” Accessed: May 05, 2026. [Online]. Available: http://163.10.22.82/OAS/recorrido_grafos/bfs__recorrido_en_amplitud.html
- [6] “Recursividad – acervo para el mejoramiento del aprendizaje de alumnos de ingeniería, en Inteligencia Artificial.” Accessed: May 05, 2026. [Online]. Available: https://virtual.cuautitlan.unam.mx/intar/?page_id=185
- [7] “What is Queue Data Structure, its Operations, Types & Applications.” Accessed: May 05, 2026. [Online]. Available: <https://intellipaat.com/blog/queue-data-structure/>

2.10 Anexos

Link del repositorio

https://github.com/Alexis112008/Recorrido_Arboles_Binarios

2.11 Código



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL
CARRERA DE SOFTWARE

